

BE AIML — Operating Systems

Topic-wise Detailed Answers

Disk Scheduling | Access Matrix | Principles of Least Privilege

FIFO • SCAN • C-SCAN • LOOK — with numerical examples & movement charts

Reference: Silberschatz, Galvin & Gagne — OS Concepts 9th Ed.

Topic	Content
Topic 1	Disk Scheduling — overview, goals, performance criteria, hardware structure
Topic 2a	FIFO (FCFS) — complete calculation with step-by-step table
Topic 2b	SCAN (Elevator) — complete calculation with step-by-step table
Topic 2c	C-SCAN — complete calculation with movement chart
Topic 2d	LOOK — complete calculation + comparison of all 4 algorithms
Topic 3	Access Matrix — concept, example, all 3 implementation methods
Topic 4	Principle of Least Privilege — definition, types, examples, OS application

Topic 1 — Disk Scheduling Algorithms

Overview | Goals | Disk Structure | Performance Metrics | Types of Algorithms

Answer

Q1. What is Disk Scheduling? Explain its need and performance criteria.

Disk Scheduling is the technique used by the OS to decide the order in which pending disk I/O requests (processes waiting to read/write disk sectors) are serviced by the disk head. Since disk access time is much slower than CPU and RAM, optimising the order of requests dramatically improves overall system performance.

Why Disk Scheduling is needed:

- Multiple processes simultaneously request disk I/O — they all arrive in the I/O queue
- If serviced in random order, the disk head moves wastefully back and forth
- Scheduling minimises head movement → reduces seek time → improves throughput
- Modern disks receive hundreds of requests per second — ordering matters enormously

Disk Access Time Components:

Component	Formula	Description	Typical time
Seek time	Time to move head to correct track	Dominant — depends on distance moved	1–20 ms
Rotational latency	Time for disk to rotate to correct sector	Half average = $1/2 \times$ rotation period	~4 ms (7200 RPM)
Transfer time	Time to transfer data from disk to buffer	Depends on data size and rotation speed	< 1 ms
Total access	Seek + Rotational + Transfer	Sum of all components	5–25 ms typical

Performance Criteria:

- **Throughput:** Number of I/O requests serviced per unit time — higher = better
- **Average Seek Time:** Average cylinder distance the head travels — lower = better
- **Average Latency:** Average wait time before a request is serviced — lower = better
- **Fairness:** Every request should be eventually serviced — no starvation
- **Response time variance:** Consistent response — critical for real-time systems

Types of Disk Scheduling Algorithms:

Algorithm	Strategy	Best for	Starvation?
FCFS / FIFO	Serve requests in arrival order	Fairness — simple systems	No (fair to all)

SSTF	Serve nearest current position first	Reduces seek time significantly	Yes — distant requests starve
SCAN (Elevator)	Move in one direction, reverse at end	General purpose — good compromise	Possible for far-end requests
C-SCAN	One direction only, wrap to start	Uniform wait times — time-shared systems	No
LOOK	Like SCAN but only to last request	Better performance than SCAN	Possible
C-LOOK	Like C-SCAN but only to last request	Best practical performance	No

Topic 2 — FIFO, SCAN, C-SCAN, LOOK — Fully Solved

Standard example: Queue=[98,183,37,122,14,124,65,67] | Head=53 | Disk=0 to 199

Problem Setup — used for ALL four algorithms:

Disk queue (requests waiting): 98, 183, 37, 122, 14, 124, 65, 67 Current disk head position: 53
Disk cylinder range: 0 to 199 Head direction (initial): Moving toward HIGHER cylinder numbers
Sorted request positions: 14, 37, 53(head), 65, 67, 98, 122, 124, 183 Goal: Calculate total head movement (sum of all cylinder distances traveled)

Algorithm 1 Q2a. FIFO (First In First Out) / FCFS — Disk Scheduling

Concept: Requests are served in the exact order they arrive in the queue. The disk head moves from one cylinder to the next regardless of position. Also called FCFS (First Come First Served).

Service Order: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Step	From	To	Direction	Distance = To - From	Running Total
1	53	98	→ Higher	98 - 53 = 45	45
2	98	183	→ Higher	183 - 98 = 85	130
3	183	37	← Lower	37 - 183 = 146	276
4	37	122	→ Higher	122 - 37 = 85	361
5	122	14	← Lower	14 - 122 = 108	469
6	14	124	→ Higher	124 - 14 = 110	579
7	124	65	← Lower	65 - 124 = 59	638
8	65	67	→ Higher	67 - 65 = 2	640

FIFO/FCFS — Head Movement Sequence — Total head movement: 640 cylinders

53	98	183	37	122	14	124	65	67
----	----	-----	----	-----	----	-----	----	----

FIFO Total Head Movement = $45+85+146+85+108+110+59+2 = 640$ cylinders
Average seek = $640 / 8 = 80.0$ cylinders per request

Advantages: Simple to implement. No starvation — every request is eventually served in order.

Disadvantages: Very poor performance — head moves wildly. Worst-case seek time. Does not consider disk position at all. Not used in production systems.

Algorithm 2 Q2b. SCAN (Elevator Algorithm) — Disk Scheduling

Concept: The disk head sweeps across the disk like an elevator — moving in one direction (serving all requests in that direction), reaching the end of the disk (cylinder 0 or 199), then reversing direction and serving requests on the return journey. Named "elevator algorithm" because it mimics how elevators work.

Direction Rule: Head starts at 53, moves toward HIGHER cylinders first.

Phase 1 — Moving UP (53 to 199): Serve all requests with cylinder > 53: 65, 67, 98, 122, 124, 183. Then reach end of disk at 199.

Phase 2 — Moving DOWN (199 to 0): Reverse direction and serve: 37, 14.

Service Order: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14

Step	From	To	Direction	Distance	Running Total	Notes
1	53	65	→ Higher	$ 65-53 = 12$	12	Phase 1: going up
2	65	67	→ Higher	$ 67-65 = 2$	14	
3	67	98	→ Higher	$ 98-67 = 31$	45	
4	98	122	→ Higher	$ 122-98 = 24$	69	
5	122	124	→ Higher	$ 124-122 = 2$	71	
6	124	183	→ Higher	$ 183-124 = 59$	130	Last request going up
7	183	199	→ Higher	$ 199-183 = 16$	146	Reach END of disk
8	199	37	← Lower	$ 37-199 = 162$	308	Phase 2: going down
9	37	14	← Lower	$ 14-37 = 23$	331	Last request going down

SCAN (Elevator) — Head Movement Sequence — Total head movement: 331 cylinders

53	65	67	98	122	124	183	199	37	14
----	----	----	----	-----	-----	-----	-----	----	----

SCAN Total Head Movement = $12+2+31+24+2+59+16+162+23 = 331$ cylinders
Average seek = $331 / 8 = 41.4$ cylinders per request (much better than FIFO)

Why better than FIFO: Head never makes wild jumps across the disk. Requests in the current direction are served before reversing. Like an efficient elevator.

Disadvantage: Requests just behind the current head position must wait for the head to go all the way to the end, reverse, and come back — long wait.

Starvation risk: Requests near the middle of the disk are served more frequently than those at the extremes (cylinders near 0 or 199).

Algorithm 3 Q2c. C-SCAN (Circular SCAN) — Disk Scheduling

Concept: C-SCAN is a variation of SCAN that provides more uniform waiting times. The head moves in ONE direction only (e.g., always toward higher cylinders). When it reaches the end of the disk (cylinder 199), it immediately returns to the BEGINNING (cylinder 0) WITHOUT servicing any requests on the return

trip, then continues in the same direction again. The disk is treated as a CIRCULAR structure.

Service Order: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 (end) → [jump to 0] → 14 → 37

Step	From	To	Direction	Distance	Running Total	Notes
1	53	65	→ Higher	$ 65-53 = 12$	12	Moving up
2	65	67	→ Higher	$ 67-65 = 2$	14	
3	67	98	→ Higher	$ 98-67 = 31$	45	
4	98	122	→ Higher	$ 122-98 = 24$	69	
5	122	124	→ Higher	$ 124-122 = 2$	71	
6	124	183	→ Higher	$ 183-124 = 59$	130	Last request going up
7	183	199	→ Higher	$ 199-183 = 16$	146	Reach disk END
8	199	0	JUMP	$ 0-199 = 199$	345	Wrap to start (counted!)
9	0	14	→ Higher	$ 14-0 = 14$	359	Resume from beginning
10	14	37	→ Higher	$ 37-14 = 23$	382	Last remaining request

C-SCAN — Head Movement Sequence — Total head movement: 382 cylinders

53	65	67	98	122	124	183	199	0	14	37
----	----	----	----	-----	-----	-----	-----	---	----	----

C-SCAN Total Head Movement = $12+2+31+24+2+59+16+199+14+23 = 382$ cylinders
Average seek = $382 / 8 = 47.75$ cylinders per request
Note: The 199-cylinder "jump" from end to start counts in total movement. However, the KEY advantage is UNIFORM waiting time for all requests.

Why C-SCAN is preferred over SCAN for time-sharing:

- SCAN: Requests near end just serviced must wait for head to go all the way and come back (~2× disk traversal)
- C-SCAN: All requests wait at most 1× full traversal — much more uniform, fair waiting times
- New requests arriving just after the head passes are served in the next complete scan
- Used in systems where predictable, fair response time is more important than raw speed

Algorithm 4 Q2d. LOOK — Disk Scheduling

Concept: LOOK is an improved version of SCAN. Instead of going all the way to the physical end of the disk (cylinder 0 or 199), the head only goes as far as the LAST REQUEST in the current direction, then immediately reverses. It "looks" ahead to see if there are any more requests before continuing — hence

the name LOOK.

Key difference from SCAN: SCAN goes to cylinder 199 even if the last request is at 183. LOOK stops at 183 (the last actual request), saving the 16-cylinder trip to 199.

Phase 1 — Moving UP to last request: From 53 → serve 65, 67, 98, 122, 124, 183 (highest request — STOP here, reverse)

Phase 2 — Moving DOWN: From 183 → serve 37, 14 (lowest request — STOP here)

Service Order: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 37 → 14

Step	From	To	Direction	Distance	Running Total	Notes
1	53	65	→ Higher	$ 65-53 = 12$	12	Phase 1: going up
2	65	67	→ Higher	$ 67-65 = 2$	14	
3	67	98	→ Higher	$ 98-67 = 31$	45	
4	98	122	→ Higher	$ 122-98 = 24$	69	
5	122	124	→ Higher	$ 124-122 = 2$	71	
6	124	183	→ Higher	$ 183-124 = 59$	130	LAST request going up → REVERSE
7	183	37	← Lower	$ 37-183 = 146$	276	Phase 2: going down
8	37	14	← Lower	$ 14-37 = 23$	299	LAST request going down → STOP

LOOK — Head Movement Sequence — Total head movement: 299 cylinders

53	65	67	98	122	124	183	37	14
----	----	----	----	-----	-----	-----	----	----

LOOK Total Head Movement = $12+2+31+24+2+59+146+23 = 299$ cylinders
Average seek = $299 / 8 = 37.4$ cylinders per request [BEST of the four!] LOOK saves 16 cylinders vs SCAN by not going to end of disk (183 to 199 avoided)

Why LOOK is better than SCAN: Eliminates unnecessary head movement past the last request to the physical edge of the disk. More practical and efficient in real systems.

COMPLETE COMPARISON — All Four Algorithms:

Algorithm	Service order	Total movement	Avg seek	Starvation?	Best use case
FIFO/FCFS	Arrival order: 98,183,37,122,14,124,65,67	640 cylinders	80.0	None — fully fair	Simple/light load

SCAN	Higher: 65,67,98,122,124,183,199 then Lower: 37,14	331 cylinders	41.4	Extremes may wait	General purpose
C-SCAN	Higher: 65..183,199→0 then continue: 14,37	382 cylinders	47.75	None — uniform	Time-sharing/servers
LOOK	Higher: 65,67,98,122,124,183 then Lower: 37,14	299 cylinders	37.4	Extremes may wait	Best performance

Winner: LOOK uses only 299 cylinders — best performance for this example. FIFO uses 640 — worst.

Topic 3 — Access Matrix

Protection Model | Matrix Structure | All 3 Implementation Methods

Long
Answer

Q3. Explain Access Matrix — concept, structure, example, and all 3 implementation methods.

What is an Access Matrix?

The Access Matrix is a general model of protection used in operating systems. It abstractly represents the protection state of a system as a matrix where:

- **Rows** represent Domains (subjects) — users, processes, or groups that can perform operations
- **Columns** represent Objects (resources) — files, devices, memory segments, processes, other domains
- **Entry** $\text{access}[i][j]$ = the set of operations that Domain i is permitted to perform on Object j

Key terms:

Term	Definition	Example
Domain	The set of (object, rights) pairs that define what a process can access	Domain D1 = {(File1:read,write), (Printer:print)}
Object	Any hardware or software resource that can be protected	File, directory, device, semaphore, process
Access rights	Specific operations a domain can perform on an object	read, write, execute, append, delete, print, wakeup
Domain switching	A process can switch from one domain to another (if allowed)	User process → OS kernel domain (via system call)

Example Access Matrix (4 Domains, 5 Objects):

Domain / Object	File F1	File F2	File F3	Printer P1	Process Q1
D1 (Student)	read	read	—	print	—
D2 (Faculty)	read,write	read,write	execute	print	wakeup
D3 (Admin)	read,write,exec	read,write,exec	read,write	print	all
D4 (OS/Kernel)	all	all	all	all	all

Reading the matrix: D2 (Faculty) can read and write File F1, can execute File F3, can print to Printer P1, and can wakeup Process Q1. Faculty has no access to Object not listed.

Three Implementation Methods:

Method 1 — Global Table

Store the entire access matrix as a single table. Each row = one (Domain, Object, Rights) triple.

Entry #	Domain	Object	Access Rights
1	D1	File F1	read

2	D1	File F2	read
3	D1	Printer P1	print
4	D2	File F1	read, write
5	D2	File F3	execute
...	...	...	...

- **Lookup:** To check if Domain *i* can access Object *j* — scan the table for entry (*i*, *j*)
- **Advantage:** Conceptually simple — one centralised table
- **Disadvantage:** Huge table for large systems. Slow lookup — must scan linearly. Cannot efficiently group objects with same access patterns.

Method 2 — Access Control Lists (ACLs) — Column-based decomposition

Each OBJECT has its own list of (Domain, Rights) pairs. The access matrix is stored column by column.

ACL for File F1: [(D1: read), (D2: read,write), (D3: read,write,exec), (D4: all)] ACL for File F2: [(D1: read), (D2: read,write), (D3: read,write,exec), (D4: all)] ACL for Printer P1: [(D1: print), (D2: print), (D3: print), (D4: all)] ACL for Process Q1: [(D2: wakeup), (D3: all), (D4: all)] Usage: When Domain D2 requests access to File F1: 1. Look up ACL of File F1 2. Search for entry with Domain D2 → found: {read, write} 3. Check if requested operation (e.g., read) is in {read, write} → YES → allow

- **Advantage:** Easy to determine who can access a file. Easy revocation — just remove entry from file's ACL. Natural fit for file systems.
- **Advantage:** Supports wildcards — (ALL users: read) or (group Faculty: write)
- **Disadvantage:** Difficult to find all rights a domain has — must scan ALL ACLs. If domain is deleted, must scan all ACLs to remove it.
- **Real systems:** UNIX/Linux file permissions (rwxrwxrwx), Windows NTFS ACLs, AWS IAM policies

Method 3 — Capability Lists — Row-based decomposition

Each DOMAIN has its own list of (Object, Rights) pairs called capabilities. A capability is an unforgeable token that grants specific rights to a specific object.

Capability list for D1 (Student): [(File F1: read), (File F2: read), (Printer P1: print)] Capability list for D2 (Faculty): [(File F1: read,write), (File F2: read,write), (File F3: execute), (Printer P1: print), (Process Q1: wakeup)] Usage: When Process running in D2 requests to write File F1: 1. OS looks up D2's capability list 2. Finds capability (File F1: read, write) 3. Checks if write is in rights → YES → grant access

- **Advantage:** Easy to determine what a process can do — just read its capability list. Fast access check. Easy to delegate capabilities.
- **Advantage:** Capabilities can be passed between processes — enables distributed systems
- **Disadvantage:** Hard to REVOKE — once a capability is given, it may be copied and shared. Must track all copies to revoke.
- **Disadvantage:** Capabilities must be protected — if an attacker can forge a capability, they gain unauthorized access
- **Real systems:** Mach OS, KeyKOS, EROS OS, modern microkernel systems

Final Comparison — All 3 Methods:

Property	Global Table	Access Control Lists	Capability Lists
Structure	Single flat table	Per-object list	Per-domain list
Storage	One table	Attached to each object	Attached to each domain
Find all rights of domain?	Scan whole table	Scan all objects' ACLs	Read domain's list (fast)
Find who can access object?	Scan whole table	Read object's ACL (fast)	Scan all domains' lists
Revocation ease	Easy — remove entry	Easy — modify object ACL	Hard — must find all copies
Scalability	Poor — huge table	Good — per-object	Good — per-domain
Real-world use	Rarely used alone	UNIX, Windows NTFS, AWS	Mach, KeyKOS, microkernel

Topic 4 — Principle of Least Privilege (POLP)

Definition | Types | Examples | OS Application | Connection to Access Matrix

Long
Answer

Q4. Explain the Principle of Least Privilege — definition, types, examples, and OS application.

Definition:

The Principle of Least Privilege (POLP) states that every subject (user, process, program, system component) must be able to access **ONLY** the information and resources that are necessary for its legitimate purpose — and nothing more. Grant the minimum permissions required to do the job.

"Every program and every user of the system should operate using the least set of privileges necessary to complete the job." — Jerome Saltzer (MIT), 1974

Simple Analogy:

Think of a hospital:

- A nurse can access patient records for their assigned ward **ONLY**
- A receptionist can schedule appointments but **CANNOT** access lab results
- A janitor can enter any room but **CANNOT** access computers
- A surgeon can operate but **CANNOT** modify billing records

Each person has exactly the access needed for their job — no more, no less. If a nurse account is compromised, the attacker can only access that ward's records, not the entire hospital database.

Three Core Types of Least Privilege:

1. Minimal Privilege (Minimal Access Rights):

- Grant the exact set of access rights needed — no more
- A text editor should not have network access or camera access
- A payroll program should read employee salary data but **NOT** modify HR records
- OS application: File system permissions — grant only read if write is not needed

2. Minimal Duration (Time-Limited Privilege):

- Elevated privileges should be granted for the shortest time necessary
- `sudo` in Linux — elevates to root for one command, not permanently
- System administrator should use admin account only when doing admin tasks
- Temporary access tokens should expire — not permanent credentials
- OS application: `sudo` timeout, session tokens, temporary capability grants

3. Minimal Attack Surface (Need-to-Know):

- Each component should know only what it needs — information compartmentalisation
- A module handling login should not have access to payment processing code
- Microservices architecture: each service accesses only its own database
- OS application: kernel modules have access only to relevant kernel subsystems

Real-World Examples with OS Context:

Scenario	Without Least Privilege	With Least Privilege
----------	-------------------------	----------------------

Web server process	Runs as root — if hacked, attacker owns the machine	Runs as "www-data" user — limited permissions, contained breach
Database connection	App uses "root" MySQL account	App uses restricted account — only SELECT on its own tables
Android app permissions	App requests contacts, camera, microphone, location on install	App requests only what it needs; user grants each specifically
OS kernel	Monolithic — all code in kernel space with full privilege	Microkernel — minimal core; drivers run in user space with limited rights
System calls	All processes can call any system call	Seccomp in Linux — filters system calls a process can make
File access	World-readable important config files	Config files chmod 600 — only owner (root) can read

Why Least Privilege is Critical — Security Benefits:

- **Limits damage:** If a process is compromised, the attacker can only access what that process was allowed to access — not the entire system
- **Reduces attack surface:** Fewer privileges = fewer ways to exploit the system
- **Slows lateral movement:** An attacker who compromises one component cannot easily pivot to other parts of the system
- **Enforces accountability:** Actions are tied to specific identities with known limited access
- **Prevents privilege escalation:** A user process cannot gain admin rights without explicit authorisation

Implementation of Least Privilege in OS:

OS Mechanism	How it enforces Least Privilege
Dual mode (user/kernel)	User processes cannot execute privileged instructions — only kernel can
File permissions (rwx)	Access granted per user/group/other — only necessary bits set
setuid/setgid bits	Program runs with specific user's privileges, not current user's
sudo / su	Temporary privilege elevation for specific commands only
Seccomp (Linux)	Restricts which system calls a process is allowed to invoke
SELinux / AppArmor	Mandatory Access Control — enforces fine-grained per-application policies
Capabilities (POSIX)	Split root privilege into fine-grained capabilities (CAP_NET_BIND_SOCKET, CAP_SYS_TIME etc.)
Sandboxing	Run programs in isolated environment with minimal host access (Chrome, Docker)
Access Control Lists	Fine-grained per-object permission control beyond basic unix permissions

Connection to Access Matrix:

The Access Matrix directly implements the Principle of Least Privilege. Each entry access[domain][object] contains only the MINIMUM rights required by that domain for that object. Empty entries (—) mean no access — the default. The matrix should be designed so that every entry contains only the rights

absolutely necessary for the system to function correctly.

Least Privilege in Access Matrix design: Rule 1: Default is NO access – start with empty matrix Rule 2: Add ONLY the rights that are explicitly needed Rule 3: Grant access to SPECIFIC objects, not broad categories Rule 4: Revoke rights as soon as they are no longer needed Rule 5: Never grant "superuser/all" unless absolutely unavoidable

Challenges and Trade-offs:

- **Usability vs Security:** Too restrictive permissions frustrate users — they may work around controls
- **Complexity:** Fine-grained permissions are hard to manage at scale — must balance granularity
- **Legacy systems:** Old applications designed to run as root are hard to redesign
- **Dynamic needs:** Permissions may need to change as user roles evolve — requires active management

Complete Summary — All Topics

Topic	Key Formulas / Rules	Best Result
FIFO/FCFS	Serve in arrival order; $WT = \text{sum of } cyl_i+1 - cyl_i $	640 cyl (worst)
SCAN	Go to end (0 or max), reverse; serves both directions	331 cylinders
C-SCAN	One direction only; wrap around; uniform wait time	382 cylinders
LOOK	Like SCAN but stop at last request; no end-of-disk travel	299 cyl (best)
Access Matrix	$\text{access}[\text{domain}][\text{object}] = \{\text{rights}\}$; default = empty = no access	3 implementations
Least Privilege	Grant min rights, min duration, min knowledge; default deny all	Foundation of security

Formula / Definition	Expression
Total head movement (any algo)	SUM of $ cylinder_i+1 - cylinder_i $ for all i
Average seek time	Total head movement / Number of requests
Need[i][j] (banker + least priv)	Need = Maximum - Allocation (apply least privilege to Maximum)
Access right check	$\text{access}[\text{current_domain}][\text{requested_object}]$ CONTAINS requested_operation?
SCAN direction rule	Serve all in current direction → reach end → reverse
C-SCAN direction rule	Serve all in one direction → reach end → jump to other end → continue
LOOK direction rule	Serve all in current direction → reach LAST request → reverse
Least privilege definition	Grant only min rights for min time needed to do specific job